IoT for Data Science
Connected Systems Foundations

BSIT Lecture Handout

Course: Data Science Program: BSIT

Sensors • Connectivity • Edge Computing • Protocols • Security

Contents

1 Introduction to the Internet of Things

Internet of Things

The Internet of Things, or IoT, is a network of physical objects that can sense, process, communicate, and sometimes act on their environment. These objects include sensors, smart appliances, vehicles, wearable devices, industrial machines, and many other embedded systems connected through local or global networks.

Why This Topic Appears in a Data Science Course

IoT is included here because connected devices are major producers of real-world data. The focus of this handout is not data science algorithms. Instead, it explains how IoT systems are built, how devices communicate, how data moves through the system, and what technical issues must be solved before analytics can happen.

1.1 Core Ideas Behind IoT

- **Sensing:** Devices observe temperature, motion, pressure, location, light, sound, or other environmental conditions.
- **Connectivity:** Devices send telemetry to gateways, platforms, or peer devices through wired or wireless links.
- **Processing:** Data may be processed on the device, at the edge, or in the cloud.
- **Actuation:** Systems can trigger a response such as opening a valve, switching a relay, or sending an alarm.
- **Management:** Devices need configuration, authentication, updates, monitoring, and retirement over time.

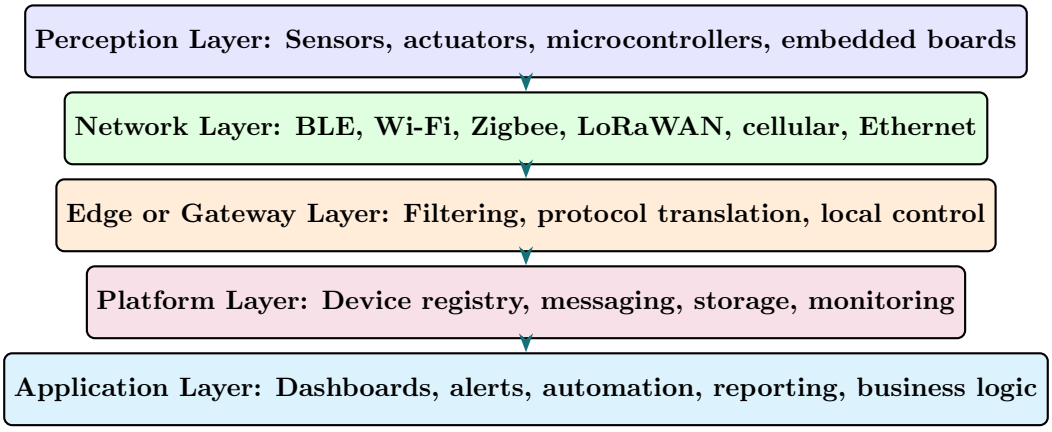
1.2 The Sense-to-Action Cycle

1.3 Common Characteristics of IoT Systems

- Large numbers of devices working together.
- Small, resource-constrained hardware with limited CPU, memory, and power.
- Continuous or periodic data generation.
- Operation in changing physical environments.
- Dependence on secure identity, reliable communication, and long-term maintainability.

2 IoT Ecosystem and Architecture

2.1 Typical IoT Architecture



2.2 Main Components in the Ecosystem

Component	Role	Examples
End device	Observes or controls the physical world	Smart meter, wearable band, soil sensor, smart bulb
Gateway	Bridges local devices to wider networks and may preprocess data	Industrial gateway, home hub, edge computer
Communication network	Carries data between nodes and services	Wi-Fi, cellular, Zigbee mesh, Ethernet
IoT platform	Handles device identity, messaging, storage, commands, monitoring	Device registry, broker, telemetry dashboard
Application	Gives value to users through control panels, alerts, and automation	Smart campus app, predictive maintenance console

2.3 Centralized and Distributed Designs

- **Centralized IoT:** Devices send most data to a cloud platform where decisions are made.
- **Distributed IoT:** More decisions happen near the devices through gateways or edge nodes.
- **Hybrid IoT:** Time-sensitive actions happen locally while long-term storage and fleet management happen centrally.

</> Architecture Choice

A smart irrigation system may measure soil moisture in the field, let a local controller decide whether to open a valve, and still send summaries to a cloud dashboard for remote supervision.

3 Sensors, Actuators, and Embedded Devices

3.1 Sensors

Sensor

A sensor is a device that converts a physical phenomenon into an electrical signal that can be measured and interpreted by a controller. Sensors are the main entry point of data into an IoT system.

Sensor Type	Measures	Typical IoT Use
Temperature sensor	Heat level	Cold-chain tracking, HVAC systems, weather stations
Motion sensor	Movement or presence	Smart lighting, intrusion detection, occupancy tracking
Pressure sensor	Force per unit area	Industrial monitoring, vehicle systems, pipelines
Light sensor	Illumination level	Street lighting, greenhouse control, phone brightness
GPS module	Geographic position	Fleet tracking, asset monitoring, navigation
Gas sensor	Air quality or chemical presence	Safety systems, pollution monitoring, smart labs

3.2 Actuators

Actuator

An actuator receives a command from the system and performs a physical action. If sensors represent input from the environment, actuators represent output back to the environment.

- Electric relays switch devices on or off.
- Motors create movement in robots, doors, pumps, and valves.
- Buzzers and lights produce alerts.
- Solenoid valves control flow in water or gas systems.

3.3 Embedded Controllers and Device Constraints

Resource Constraints

Unlike laptops or servers, many IoT devices have very limited memory, battery capacity, bandwidth, and processing power. Designers must therefore be careful about code size, communication overhead, power use, and update strategy.

- Microcontrollers
- Single-board computers
- Real-time operating systems
- Battery-powered nodes
- Sleep and wake cycles
- Local buffering

3.4 Calibration and Signal Quality

Garbage In, Garbage Out

If a sensor is poorly calibrated, badly placed, electrically noisy, or physically damaged, every higher layer of the IoT system is affected. Reliable IoT starts with correct sensing, not with dashboards.

4 Connectivity and Communication Protocols

4.1 Connectivity Options

- **Wi-Fi:** High throughput for homes, buildings, and campuses, but relatively high power use.
- **Bluetooth Low Energy (BLE):** Short-range, low-power communication for wearables and personal devices.
- **Zigbee and Thread:** Low-power mesh networking for smart homes and building automation.
- **LoRaWAN:** Long-range, low-bandwidth communication for remote sensing and smart agriculture.
- **Cellular:** Wide-area coverage for vehicles, logistics, and mobile deployments.
- **Ethernet:** Stable wired option for industrial and fixed installations.

4.2 Application Protocols

Protocol	Pattern	Strength	Typical Use
MQTT	Publish and subscribe	Lightweight messaging over unreliable links	Telemetry from constrained devices
HTTP	Request and response	Simple and widely supported	Web APIs, configuration portals
CoAP	Request and response	Designed for constrained devices and UDP transport	Low-power sensor nodes
AMQP	Message queues	Reliable enterprise messaging	Industrial integration, backend systems

4.3 Topics, Messages, and Payloads

Message Design

IoT systems do not only need connectivity. They also need a message design: what is sent, how often it is sent, which device sent it, how the receiver interprets it, and what happens when messages arrive late, duplicated, or out of order.

MQTT Topic Example

An MQTT topic such as `campus/buildingA/room204/temperature` lets subscribers receive only the telemetry they need. Topic naming helps organize large fleets of devices.

4.4 Network Tradeoffs

- Long range often means lower data rate.
- Low power often means small payloads and less frequent transmission.
- High throughput often requires more energy and stronger infrastructure.
- Reliability may require acknowledgments, retries, buffering, or redundant paths.

5 Edge, Fog, and Cloud in IoT

5.1 Where Processing Happens

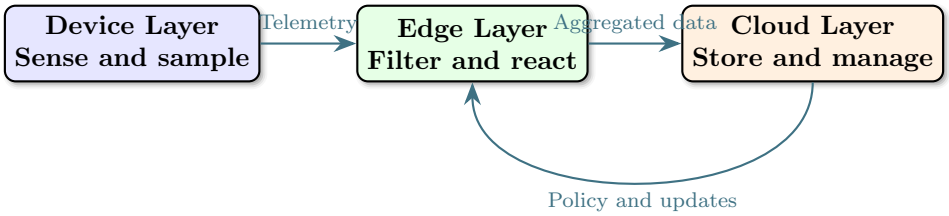
Edge Computing

Edge computing means processing data close to where it is generated, such as on the device itself or on a nearby gateway. This reduces latency, saves bandwidth, and can improve privacy.

Fog Computing

Fog computing is an intermediate model in which processing is distributed across local or regional infrastructure between the device layer and the cloud. It is useful when many nearby devices need coordination.

5.2 Edge-to-Cloud Flow



5.3 Why Edge Computing Matters in IoT

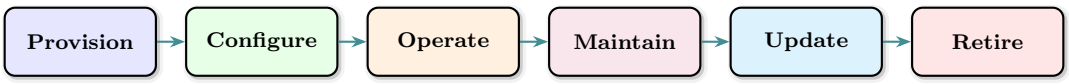
- Local decisions can happen even when internet connectivity is weak or absent.
- Bandwidth costs are reduced because only summaries or events need to be transmitted.
- Sensitive raw data may stay closer to where it was generated.
- Time-critical systems such as robotics or safety alarms need low latency.

5.4 When Cloud Still Matters

- Fleet-wide device management.
- Long-term storage and compliance archives.
- Global dashboards and multi-site visibility.
- Centralized software update control.
- Integration with enterprise applications and identity services.

6 Device Lifecycle and IoT Data Flow

6.1 From Provisioning to Retirement



6.2 Lifecycle Stages

- **Provisioning:** Register the device, assign credentials, and define identity.
- **Configuration:** Set sampling intervals, thresholds, network settings, and policy.
- **Operation:** Collect telemetry, receive commands, and monitor status.
- **Maintenance:** Replace batteries, recalibrate sensors, and repair hardware faults.
- **Firmware updates:** Deliver secure over-the-air updates to fix bugs or add features.
- **Retirement:** Revoke credentials, archive records, and safely remove the device from service.

6.3 Telemetry, Events, and Commands

Message Type	Direction	Purpose
Telemetry	Device to platform	Periodic measurements such as temperature or battery level
Event	Device to platform	Report something meaningful such as a door opened or threshold exceeded
Command	Platform to device	Request an action such as restart, set fan speed, or open valve
Status report	Device to platform	Share health information such as uptime or firmware version

6.4 Digital Twins

Digital Twin

A digital twin is a virtual representation of a physical device or asset. It stores the device state, desired configuration, and operational history so software can reason about the real object without direct physical access.

7 Security, Privacy, and Reliability in IoT

7.1 Why IoT Security Is Difficult

A Larger Attack Surface

IoT security is harder than ordinary application security because the system includes hardware, firmware, radio communication, network services, cloud services, physical access risks, and long-lived devices that may remain deployed for years.

7.2 Common Security Risks

- Default passwords or weak authentication.
- Unencrypted communication over insecure channels.
- Unsigned firmware updates or poorly protected boot processes.
- Insecure APIs exposing device control or telemetry.
- Physical tampering with field devices.
- Poor key storage on constrained hardware.

7.3 Essential Security Controls

- Unique device identity for every node.
- Mutual authentication between device and platform.
- Encryption in transit and secure storage for secrets.
- Secure boot and signed firmware updates.
- Least-privilege access for applications and operators.
- Logging, anomaly detection, and incident response procedures.

7.4 Privacy and Ethics

Responsible Collection

Many IoT systems observe people, movement, health, or behavior. Designers must therefore think about consent, data minimization, lawful use, retention limits, and the consequences of continuous monitoring.

7.5 Reliability and Maintenance

- Plan for packet loss, dead batteries, sensor drift, and unstable connectivity.
- Use retries, buffering, heartbeats, and watchdog timers where appropriate.
- Expect hardware failure and design for replacement.
- Measure uptime, signal quality, message delay, and battery health.

8 IoT Use Cases, Challenges, and BSIT Skills

8.1 Representative IoT Use Cases

- **Smart homes:** Lighting, security, energy monitoring, and appliance control.
- **Industrial IoT:** Machine monitoring, predictive maintenance, and process automation.
- **Smart agriculture:** Soil moisture sensing, weather monitoring, and irrigation control.
- **Healthcare IoT:** Wearables, remote patient monitoring, and medication tracking.
- **Smart cities:** Parking systems, traffic sensors, environmental monitoring, and smart bins.
- **Smart campuses:** Room occupancy, lab condition monitoring, access control, and utility tracking.

8.2 Mini Case Study: Smart Classroom Monitoring

</> Scenario

A campus installs environmental sensors in classrooms to track temperature, humidity, air quality, and occupancy. A local gateway collects the readings, filters noisy values, and forwards summaries to a platform dashboard. Facility staff receive alerts if ventilation quality drops or a room is over-occupied.

IoT concepts shown by this example:

- Embedded sensors at the device layer.
- Local gateway processing at the edge layer.
- Lightweight telemetry messaging to the platform.
- Device management for calibration and firmware updates.
- Security controls for access to occupancy data and control interfaces.

8.3 Technical Challenges in Real IoT Projects

Challenge	Why It Matters
Power management	Battery-powered devices must last for months or years
Interoperability	Devices from different vendors may not share the same standards
Network instability	Remote and mobile deployments may lose connectivity often
Security maintenance	Weak update processes leave devices exposed for long periods
Scalability	Device fleets can grow from tens to thousands of nodes
Physical environment	Heat, dust, vibration, and moisture affect hardware quality

8.4 Skills for BSIT Students

- Basic electronics and sensing
- Embedded programming fundamentals
- Networking and IP basics
- MQTT and REST API understanding
- Linux and gateway setup
- Security principles for devices
- Monitoring and troubleshooting
- Documentation and system diagrams

9 Key Takeaways and Review Questions

9.1 Key Takeaways

- IoT connects physical objects to digital systems through sensing, communication, processing, and actuation.

- A complete IoT solution includes devices, networks, edge or gateway services, platforms, and applications.
- IoT design depends heavily on hardware limits, communication tradeoffs, update strategy, and field maintenance.
- Security and privacy are core design requirements, not optional additions.
- Understanding IoT concepts helps BSIT students work with real-world data-producing systems before any analytics stage begins.

9.2 Review Questions

1. What is the difference between a sensor and an actuator?
2. Why do many IoT systems use lightweight protocols such as MQTT?
3. How does edge computing improve some IoT deployments?
4. What happens during the provisioning stage of the device lifecycle?
5. Why is IoT security more difficult than ordinary web application security?
6. Name two tradeoffs that appear when choosing a communication technology for IoT.

</> Short Reflection Task

Design a simple IoT solution for your school, home, or community. Identify the sensor, the network connection, whether processing happens at the edge or in the cloud, one possible actuator, and one security control you would require before deployment.

End of Handout

IoT for Data Science — BSIT Program